AI Startup Intern Assignment

Build a Production-Oriented AI Research Agent

Context

We are building production-grade agentic AI systems. This assignment is designed to assess how you think about agent workflows, tool use, reliability, evaluation, and engineering tradeoffs.

This is not a prompt-engineering exercise and not a UI-design exercise. We are more interested in your system design and implementation judgment than a polished frontend.

Objective

Build an AI agent that can answer a research-style question by:

- understanding the request,
- planning how to solve it,
- using tools to gather information,
- synthesizing a structured answer,
- handling failure cases,
- showing how you would evaluate and improve the system.

Example Queries

- Compare the top 3 open-source vector databases for a startup building RAG products.
- Find 5 Indian B2B SaaS startups in HR tech and summarize their positioning.
- Research the pros and cons of using a multi-agent architecture for customer support automation.
- Compare different approaches to adding memory in an AI support agent.

What You Need to Build

Create a small but production-minded agentic system with the following flow:

1. Accept a user question.
2. Create a plan to answer it.
3. Decide which tools to use.

4. Gather and filter information.
5. Produce a structured final response.
6. Report confidence, assumptions, and limitations.

Minimum Requirements

- A working agentic workflow
- At least 2 tools
- A visible planning or orchestration structure
- Structured final output
- Basic failure handling
- A short evaluation framework
- A README explaining architecture and tradeoffs

Functional Expectations

Your system should demonstrate:

- Tool selection based on the task
- Retrieval of relevant information
- Filtering of noisy or weak information
- Final-answer synthesis with sources
- Clear output formatting

The final answer should include:

- User question
- Short answer
- Key findings
- Sources used
- Confidence level
- Limitations or assumptions
- Suggested next steps

Non-Functional Expectations

We want to see whether you think about production concerns such as:

- timeout and retry behavior,
- malformed tool output,
- low-quality or irrelevant sources,
- hallucination risk,
- prompt injection awareness,
- deterministic output structure,
- logging or tracing,
- cost and latency tradeoffs.

You do not need to solve every production concern fully. We care more about whether you identify the risks and make sensible tradeoffs.

> **Recommended scope:** This is a medium-difficulty take-home assignment. Expected effort is 6 to 10 hours. Build a focused system, not a large platform.

Preferred Tech Stack

You may use any stack, but common choices include:

- Python or TypeScript
- OpenAI API or another LLM provider
- OpenAI SDK, LangGraph, PydanticAI, or a simple custom orchestration layer
- FastAPI, Streamlit, or a simple CLI

We value clarity more than framework complexity.

Guidance

- Start simple. A clean single-agent design is better than an overbuilt system.
- Be explicit about assumptions.
- Explain why an agent is appropriate for this task.
- Show how your tool orchestration works.
- Think about failure modes, not just the happy path.
- Use structured output where possible.
- If something is incomplete, call it out honestly and explain what you would do next.

Suggested Resources

You are welcome to use any resources you find useful. The following topics are especially relevant:

- LLM tool calling
- Structured outputs and schema validation
- ReAct-style and planner-executor agent patterns
- Grounding and citations
- Prompt injection and tool safety
- Evaluation of LLM systems
- Logging, tracing, and observability

Useful libraries and tools may include:

- OpenAI SDK
- LangGraph
- Pydantic or Zod
- FastAPI or Streamlit
- Langfuse, Helicone, or OpenTelemetry

Optional Stretch Goals

- citation grounding,
- evaluator or judge step,
- caching,
- memory for follow-up questions,
- async tool execution,
- guardrails for low-confidence answers,
- comparison with a non-agentic baseline.

Submission Instructions

Please submit the following:

1. A source code repository
2. A README containing setup instructions, architecture overview, design decisions, known limitations, and future improvements

3. A short demo video, 3 to 5 minutes, showing one happy path and one failure or edge case
4. A brief evaluation document with at least 5 test queries, expected behavior, and what worked well and what did not
5. Two or three sample outputs from your system

Your README Should Answer

- Why did you choose an agentic approach here?
- What tools did you use, and why?
- How does the system handle bad tool results?
- How do you reduce hallucinations?
- How would you make this production-ready?
- What would you monitor in production?
- What were your key tradeoffs?

Evaluation Criteria

- Problem understanding
- Architecture clarity
- Tool orchestration
- Code quality
- Reliability thinking
- Evaluation quality
- Product and engineering judgment
- Communication quality

What Strong Submissions Usually Show

- A clear and simple architecture
- Good tool use and decision logic
- Structured, source-backed output
- Thoughtful handling of failure cases
- Practical evaluation discipline
- Honest discussion of limitations and tradeoffs

What Weak Submissions Usually Show

- A thin chat wrapper with no real agent workflow
- No meaningful tool use
- No source grounding
- No handling of failure cases
- No evaluation discipline
- Excessive framework complexity with limited clarity

How We Recommend You Work

1. Define the input and output format.
2. Implement a simple planner.
3. Add 2 tools.
4. Add answer synthesis.
5. Add failure handling.
6. Run test queries and document what breaks.
7. Write up tradeoffs and next steps.

Submission Format

Please send your repository link, demo video link, evaluation notes, and any setup instructions needed to run your project.

Final Note

We are not looking for perfection. We are looking for clear thinking, practical engineering judgment, and signs that you can help build reliable AI systems in the real world.